

Aprendizaje de Máquinas

Control 3 Otoño 2025

Índice de Contenidos

1. Control 3 Otoño 2025	1
-------------------------	---

Índice de Tablas

1. Dataset para predicción de floración de una planta.	14
2. Tabla 2: Tabla de resultados aproximados.	15

1. Control 3 Otoño 2025

P1. (55 %) Preguntas Conceptuales

De las siguientes preguntas, debe elegir un máximo de **seis (6)** a contestar, escogiendo **al menos una (1)** de cada tópico. Sea breve y formal en sus respuestas, puede apoyarse de dibujos.

SVM con Kernel

(a) En un SVM con kernel RBF, $K(\vec{x}, \vec{x}') = \exp(-\gamma \|\vec{x} - \vec{x}'\|^2)$, ¿cómo afecta el hiperparámetro γ a la frontera de decisión? Relacione valores muy grandes y muy pequeños de γ con el dilema sesgo-varianza y la complejidad del modelo.

Respuesta:

El hiperparámetro $\gamma > 0$ controla el alcance de influencia de un solo ejemplo de entrenamiento sobre la frontera de decisión. En particular:

- **Valores muy pequeños de γ ($\gamma \rightarrow 0$):** el kernel RBF se vuelve muy ancho, de modo que todos los puntos de datos se consideran casi igualmente cercanos entre sí. La frontera de decisión resultante será muy suave y con poca complejidad, aproximándose a un modelo lineal en muchos casos. Esto puede inducir **alto sesgo** y **baja varianza**, ya que el modelo subestima la complejidad real de los datos y puede no ajustarse bien a patrones no lineales.
- **Valores muy grandes de γ ($\gamma \rightarrow \infty$):** el kernel se vuelve muy estrecho, de modo que cada punto solo influye en su vecindad inmediata. La frontera de decisión se adapta estrechamente a los puntos de entrenamiento, capturando incluso ruido local. Esto genera un **bajo sesgo** pero **alta varianza**, es decir, un modelo muy complejo que puede sobreajustarse al conjunto de entrenamiento.

En resumen, γ controla la **complejidad del modelo**: valores pequeños generan fronteras simples (alto sesgo), y valores grandes generan fronteras muy complejas (alta varianza).

(b) En un SVM con Kernel, el problema de optimización se resuelve comúnmente a través de su formulación dual:

$$\max_{\boldsymbol{\alpha}} W(\boldsymbol{\alpha}) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j K(\vec{x}_i, \vec{x}_j)$$

sujeto a:

$$\sum_{i=1}^N \alpha_i y_i = 0 \quad \text{y} \quad 0 \leq \alpha_i \leq C, \quad \forall i = 1, \dots, N$$

donde $K(\vec{x}_i, \vec{x}_j)$ es la función kernel que opera sobre los datos de entrada y C es el parámetro de regularización. Interprete el rol del parámetro de regularización $C > 0$. ¿Cómo afecta a la solución (y a la frontera de decisión resultante) un valor de $C \rightarrow \infty$ versus un valor de $C \rightarrow 0$ en términos de la complejidad del modelo y la tolerancia a errores de clasificación en el conjunto de entrenamiento? Relacione esta elección con el dilema sesgo-varianza.

Respuesta:

En la formulación dual del SVM con kernel, el parámetro de regularización $C > 0$ controla la tolerancia del modelo a los errores de clasificación en el conjunto de entrenamiento:

- **C grande** ($C \rightarrow \infty$): el modelo penaliza fuertemente los errores de clasificación. Esto conduce a una frontera que intenta clasificar correctamente todos los puntos de entrenamiento, aumentando la complejidad del modelo y reduciendo el sesgo, pero puede generar **alta varianza** si los datos contienen ruido, ya que el modelo se sobreajusta.
- **C pequeño** ($C \rightarrow 0$): el modelo permite que se produzcan errores en el conjunto de entrenamiento sin penalización significativa. La frontera de decisión se vuelve más suave y simple, lo que reduce la complejidad del modelo, aumenta el sesgo y disminuye la varianza. El modelo tiende a generalizar mejor frente a ruido, pero puede subajustar los datos.

En términos del dilema sesgo-varianza, C regula un balance entre **ajuste estricto a los datos** (baja tolerancia a errores, baja sesgo, alta varianza) y **ajuste más relajado** (alta tolerancia a errores, alto sesgo, baja varianza).

(c) El truco del kernel permite calcular el producto interno en un espacio de características de alta dimensión, $\langle \phi(\vec{x}_i), \phi(\vec{x}_j) \rangle$, usando una función kernel $K(\vec{x}_i, \vec{x}_j)$. La función de decisión de un Kernel SVM es:

$$f(\vec{x}_*) = \sum_{i \in SV} \alpha_i^* y_i K(\vec{x}_i, \vec{x}_*) + b^*$$

Explique por qué esta formulación solo depende de los **vectores de soporte (SV)**. ¿Qué implicación computacional tiene esto, especialmente cuando el conjunto de entrenamiento es muy grande pero el número de vectores de soporte es pequeño?

Respuesta:

La función de decisión de un Kernel SVM está dada por:

$$f(\vec{x}_*) = \sum_{i \in SV} \alpha_i^* y_i K(\vec{x}_i, \vec{x}_*) + b^*.$$

- Solo los **vectores de soporte (SV)** contribuyen a la función de decisión porque para cualquier punto que no sea SV, el multiplicador $\alpha_i^* = 0$. Esto significa que solo los puntos que se encuentran en los márgenes o que se clasifican incorrectamente determinan la frontera de decisión.
- **Implicación computacional:** si el conjunto de entrenamiento es muy grande pero el número de vectores de soporte es pequeño, la evaluación de la función de decisión para nuevos datos es eficiente, ya que solo se necesita calcular el kernel respecto a los SV, no a todos los puntos de entrenamiento.

Esto permite que SVM con kernel escale mejor en predicción, aunque el entrenamiento pueda ser costoso.

Reducción de Dimensionalidad

(a) Compare y contraste **PCA** y **LDA**. Enfóquese en: (1) el objetivo de cada algoritmo (supervisado vs. no supervisado), (2) el criterio que optimizan para encontrar las proyecciones, y (3) el número máximo de dimensiones que cada uno puede generar.

Respuesta:

(1) Objetivo: supervisado vs. no supervisado

PCA es un método **no supervisado**. No utiliza etiquetas de clase, y su objetivo es encontrar direcciones que capturen la **máxima varianza** posible en los datos. Busca describir la estructura geométrica global del conjunto sin considerar categorías.

LDA es un método **supervisado**. Utiliza las etiquetas de clase y su objetivo es encontrar proyecciones que **maximicen la separabilidad entre clases**. Busca un espacio donde las clases queden lo más separadas posible.

(2) Criterio que optimizan

PCA encuentra vectores dirección que **maximizan la varianza total**. Formalmente, el primer componente principal maximiza

$$\mathbf{w}^\top S_T \mathbf{w},$$

donde S_T es la matriz de covarianza total. No utiliza información sobre clases.

LDA busca direcciones que maximicen la **razón de Fisher**, es decir,

$$J(\mathbf{w}) = \frac{\mathbf{w}^\top S_B \mathbf{w}}{\mathbf{w}^\top S_W \mathbf{w}},$$

donde S_B es la matriz de dispersión entre clases y S_W la matriz de dispersión dentro de clases. El objetivo es maximizar la distancia entre los centroides de las clases mientras se minimiza la variabilidad interna de cada clase.

(3) Número máximo de dimensiones

PCA puede generar hasta $\min(n - 1, p)$ componentes principales, y en la práctica se considera que puede producir hasta p componentes, donde p es el número de variables originales.

LDA solo puede generar a lo más $K - 1$ **dimensiones**, donde K es el número de clases. Esto se debe a que la matriz S_B tiene rango máximo $K - 1$. Por ejemplo, con tres clases solo se pueden obtener dos discriminantes lineales.

Resumen

Característica	PCA	LDA
Naturaleza	No supervisado	Supervisado
Objetivo	Maximizar varianza total	Maximizar separabilidad entre clases
Criterio	Varianza	Razón S_B/S_W
Dimensiones máximas	Hasta p	Hasta $K - 1$

(b) Contraste los métodos de reducción de dimensionalidad **lineales** (ej. PCA) y **no lineales** (ej. Isomap, t-SNE), en términos de sus supuestos sobre la **geometría** de los datos y el tipo de **estructura** (varianza global vs. relaciones locales) que buscan preservar.

Respuesta:

Los métodos lineales asumen que la estructura esencial de los datos puede capturarse adecuadamente mediante **combinaciones lineales** de las variables originales. Bajo este supuesto, los datos se consideran contenidos (aproximadamente) en una **subvariedad lineal** o un **hiperplano** de menor dimensión dentro del espacio original.

- **Supuesto geométrico:** La estructura relevante es **global y lineal**. Se asume que la dirección de máxima variabilidad es informativa y que las relaciones importantes pueden describirse mediante vectores en un espacio euclidiano lineal.
- **Estructura preservada:** Métodos como PCA buscan preservar principalmente la **varianza global**. Esto significa que intentan conservar las direcciones donde los datos presentan mayor dispersión, incluso si estas direcciones no reflejan relaciones locales entre puntos cercanos.
- **Limitación:** Si los datos están distribuidos sobre una **variedad no lineal** (por ejemplo, una hélice, un manifold curvo o una forma en S), los métodos lineales no podrán revelar correctamente dicha estructura.

Los métodos no lineales relajan el supuesto de linealidad y asumen que los datos se encuentran sobre una **variedad curvada de baja dimensión** embebida en un espacio de mayor dimensión. Su objetivo es recuperar esta geometría intrínseca.

- **Supuesto geométrico:** Los datos viven en un **manifold no lineal**. La distancia relevante no es necesariamente la euclidiana global, sino distancias **geodésicas** (Isomap) o relaciones **locales de vecindad** (t-SNE).
- **Estructura preservada:**
 - **Isomap** busca preservar las **distancias geodésicas globales**. Esto permite desplegar la forma global de un manifold no lineal manteniendo sus distancias internas.
 - **t-SNE** busca preservar las **relaciones locales**, es decir, mantener juntos a puntos que son vecinos cercanos en el espacio original. t-SNE no intenta preservar estructura global, sino enfatizar la **cohesión local** (afinidad entre puntos).
- **Ventaja:** Pueden descubrir estructuras curvas o agrupaciones complejas que métodos lineales no detectan.
- **Limitación:** No conservan necesariamente la varianza global ni las distancias absolutas; pueden distorsionar la forma global del espacio.

Resumen

Aspecto	Lineales (PCA)	No lineales (Isomap, t-SNE)
Supuesto geométrico	Subespacio lineal	Manifold curvo no lineal
Estructura preservada	Varianza global	Distancias geodésicas o relaciones locales
Tipo de relaciones	Globales	Locales (t-SNE) o globales no lineales (Isomap)
Limitación	Incapaz ante curvatura	Distorsión global posible (t-SNE)

(c) Al visualizar datos con **t-SNE**, ¿por qué **no** es correcto interpretar las **distancias relativas** entre clústeres bien separados en el mapa de baja dimensión? ¿Qué aspecto del algoritmo causa este efecto?

Respuesta:

Por qué no deben interpretarse las distancias entre clústeres

En t-SNE:

- La posición **absoluta** de los clústeres en la proyección no tiene significado geométrico.
- La **distancia entre clústeres** puede ser arbitraria: dos clústeres pueden aparecer muy separados incluso si, en el espacio original, no lo están tanto.
- El algoritmo está diseñado para preservar **relaciones locales**, no la estructura global del espacio.

Por tanto, t-SNE es útil para identificar grupos compactos y su forma interna, pero **no** para estudiar:

- distancias entre clústeres,
- ángulos entre grupos,
- posiciones relativas globales,
- si un clúster está “cerca” o “lejos” de otro.

¿Qué parte del algoritmo causa este efecto?

El efecto proviene de dos elementos fundamentales del funcionamiento de t-SNE:

1. **t-SNE preserva probabilidades de vecindad locales:** En el espacio original, t-SNE construye distribuciones de probabilidad que modelan la **probabilidad de que un punto sea vecino de otro**. En la proyección, el objetivo es reconstruir **únicamente esas relaciones locales**, no las distancias globales.
2. **El uso de una distribución t-Student en el espacio reducido:** t-SNE usa una distribución **con colas pesadas** (t-Student) para modelar distancias en baja dimensión. Esto:
 - exagera la separación entre grupos lejanos,
 - reduce la atracción entre clústeres distintos,
 - permite que los clústeres se dispersen más de lo que realmente están.

Combinados, estos puntos implican que la estructura global en t-SNE es **distorsionada a propósito**. El algoritmo solo se preocupa de que “Los vecinos sigan siendo vecinos” sin ningún compromiso con la posición relativa de grupos distintos.

Clustering

(a) Describa el algoritmo **DBSCAN** y discuta cómo sus parámetros ϵ (**eps**) y **minPts** controlan la detección de clústeres de forma arbitraria y la identificación de ruido. Compare sus supuestos y tipo de clústeres recuperables con los de **k-Means**.

Respuesta:

DBSCAN es un método de clustering basado en densidad que agrupa puntos que están densamente conectados en el espacio de características y marca como *ruido* (outliers) las regiones de baja densidad. A grandes rasgos, el algoritmo opera así:

1. Para cada punto p en el conjunto de datos, calcular la **vecindad ϵ -vecina** $N_\epsilon(p) = \{q \mid \text{dist}(p, q) \leq \epsilon\}$.
2. Clasificar puntos según el tamaño de su vecindad:
 - **Core point (punto núcleo):** $|N_\epsilon(p)| \geq \text{minPts}$.
 - **Border point (punto frontera):** $|N_\epsilon(p)| < \text{minPts}$ pero pertenece a la vecindad ϵ de algún punto núcleo.
 - **Noise (ruido):** no es núcleo ni frontera.
3. Construir clústeres conectando transitivamente puntos núcleo que están a distancia $\leq \epsilon$, y agregando a cada clúster los puntos frontera alcanzables desde esos núcleos.

Rol de los parámetros ϵ (**eps**) y **minPts**

Estos dos parámetros controlan la definición de *densidad suficiente* y, por tanto, la forma y cantidad de clústeres hallados:

- ϵ (radio de vecindad): determina la escala espacial a la cual se mide densidad.
 - **Si ϵ es muy pequeño:** pocas vecindades alcanzan **minPts** $\Rightarrow$ muchos puntos quedan etiquetados como ruido; se obtienen clústeres pequeños o ninguno.
 - **Si ϵ es muy grande:** vecindades grandes conectan regiones distintas $\Rightarrow$ se pueden unir clústeres separados en uno solo (subagregación).
- **minPts** (punto mínimo para considerar un núcleo): controla cuán densa debe ser una región para ser considerada un clúster. Reglas prácticas:
 - $\text{minPts} \geq d + 1$ (donde d es la dimensionalidad) es una recomendación mínima.
 - Valores mayores hacen que solo regiones más densas sean núcleos (más puntos serán ruido), valores pequeños permiten detectar clústeres muy pequeños pero aumentan sensibilidad al ruido.

Efecto combinado: la pareja $(\epsilon, \text{minPts})$ define la *densidad mínima* requerida. Ajustes distintos pueden producir particiones muy diferentes (detección de clústeres arbitrarios, fusión de clústeres, o marcado de muchos puntos como ruido). Por ello *DBSCAN es sensible a la elección de parámetros*, especialmente cuando existen densidades heterogéneas en los datos.

Supuestos geométricos y tipos de clústeres recuperables

- **DBSCAN:** asume que los clústeres se manifiestan como regiones conectas de *alta densidad* separadas por regiones de *baja densidad*. Puede recuperar *formas arbitrarias* (no necesariamente convexas ni esféricas): cadenas, anillos, estructuras no lineales. Detecta automáticamente ruido/outliers. Sin embargo:
 - Tiene dificultades con *densidades heterogéneas*: un único par $(\epsilon, \text{minPts})$ puede no ser apropiado para clústeres con densidades muy distintas.
 - En dimensiones altas su sensibilidad a la métrica hace que funcione peor.
- **k-Means:** asume clústeres *esféricos* (o más precisamente, que los clústeres son regiones convexas centradas en un prototipo/centroide) y busca particionar el espacio en k grupos que minimicen la suma de distancias al cuadrado dentro de cada clúster.

Comparación DBSCAN vs k-Means (resumen)

Aspecto	DBSCAN	k-Means
Supuesto geométrico	Regiones de alta densidad separadas por baja densidad; formas arbitrarias	Clústeres aproximadamente esféricos/convexos alrededor de centroides
Número de clústeres	Se determina automáticamente (depende de ϵ, minPts)	Debe fijarse k a priori
Ruido / outliers	Detecta explícitamente ruido	No modela ruido; cada punto se asigna a algún clúster
Sensibilidad a escala	Alto (distancias)	Alto (distancias)
Densidades variables	Mal rendimiento si densidades muy heterogéneas	Puede separar grupos de distinta densidad si son convexas y bien separados (pero puede fallar si formas no esféricas)
Forma de clúster recuperable	Arbitraria (no convexa)	Esférica/convexa (no detecta formas complejas)
Complejidad	$O(n \log n)$ con estructura espacial; peor $O(n^2)$ sin índices	$O(nkt)$ por iteración (t iteraciones), generalmente rápido para k moderado

(b) La función objetivo que **k-Means** busca minimizar es la Suma de Cuadrados Intra-clúster (WCSS), basada en la distancia Euclidiana al cuadrado: $J = \sum_{k=1}^K \sum_{\vec{x}_i \in C_k} \|\vec{x}_i - \boldsymbol{\mu}_k\|^2$. Explique por qué el pre-procesamiento de los datos, como la estandarización es a menudo un paso crucial antes de aplicar k-Means. ¿Qué pasaría si una característica tuviera una magnitud (ej. varianza) mucho mayor que las otras? ¿Cómo influiría esto en la forma de los clústeres y en qué característica dominaría el cálculo de la distancia?

Respuesta:

¿Por qué estandarizar / escalar antes de k-Means?

- Si una característica j tiene varianza mucho mayor que las demás, sus diferencias $(x_{ij} - \mu_{kj})$ serán típicamente más grandes y, al elevarse al cuadrado, *dominarán* la suma de cuadrados. En la práctica esto hace que k-Means agrupen principalmente según esa característica, ignorando variaciones relevantes en las demás.
- **Forma de los clústeres:** La métrica euclidiana pondera cada eje igualmente solo si las características están en escalas comparables. Si no, la forma efectiva de los clústeres se distorsiona: los clústeres aparecerán *aplanados* o *extendidos* a lo largo de la(s) dimensión(es) de mayor varianza, y la partición tenderá a ser casi unidimensional.
- **Sesgo en la localización de centroides:** En el algoritmo, los centroides $\boldsymbol{\mu}_k$ se calculan como medias por componente. Características de gran escala arrastran las medias y, por ende, la asignación de puntos a clusters.

Resumen Estandarizar (o en general escalar) los datos es a menudo **crítico** antes de aplicar k-Means porque la función objetivo se basa en distancias euclidianas al cuadrado: una característica con varianza mucho mayor dominará el cálculo de distancia, sesgará la forma de los clústeres y hará que las particiones dependan más de la unidad/escala que de la estructura real de los datos.

(c) En el **clustering jerárquico**, el método de **Ward** busca minimizar el incremento de la varianza total intra-clúster en cada fusión. La función objetivo es minimizar $\Delta J(C_A, C_B) = \frac{|C_A||C_B|}{|C_A|+|C_B|} \|\mu_A - \mu_B\|^2$. ¿Qué tipo de clústeres tiende a favorecer este criterio? Compare esta tendencia con la del **enlace simple (single linkage)**.

Respuesta:

Criterio de Ward y su interpretación: El método de Ward es un enfoque de **clustering jerárquico aglomerativo** que, en cada paso, fusiona los dos clústeres cuya unión produce el *menor incremento* en la varianza intra-clúster total. Su función objetivo es:

$$\Delta J(C_A, C_B) = \frac{|C_A||C_B|}{|C_A|+|C_B|} \|\mu_A - \mu_B\|^2,$$

donde $|C_A|$ y $|C_B|$ son los tamaños de los clústeres y μ_A, μ_B sus centroides.

Este criterio penaliza fuertemente la unión de clústeres cuyos centroides estén alejados, especialmente si los dos clústeres son grandes.

¿Qué tipo de clústeres favorece Ward?

Ward tiende a producir clústeres:

- **Compactos:** minimiza la varianza interna, promoviendo grupos “apretados” alrededor del centroide.
- **Esféricos o aproximadamente convexos:** al depender de la distancia euclidiana al centroide, favorece clústeres que se asemejen a hiperesferas o regiones convexas.
- **De tamaño relativamente balanceado:** la fracción $\frac{|C_A||C_B|}{|C_A|+|C_B|}$ crece cuando ambos clústeres tienen tamaños similares, por lo que unir dos clústeres grandes puede aumentar mucho la función objetivo; en cambio, un clúster pequeño se fusiona fácilmente con uno grande si los centroides están cercanos.
- **Resistentes al encadenamiento (anti-chaining):** Ward evita crear clústeres alargados o basados en conexiones puntuales (lo contrario al single linkage).

Comparación con el enlace simple (single linkage)

El método de **single linkage** define la distancia entre clústeres como la mínima distancia entre cualquier par de puntos (uno en cada clúster):

$$d_{SL}(C_A, C_B) = \min_{\mathbf{x} \in C_A, \mathbf{y} \in C_B} \|\mathbf{x} - \mathbf{y}\|.$$

Este enfoque tiene propiedades muy distintas de Ward:

- **Favorece clústeres alargados e irregulares:** Con que dos clústeres tengan un par de puntos cercanos, se fusionan, aunque la estructura global sea muy dispersa.
- **Sufre del efecto de encadenamiento (chaining):** forma clústeres extensos conectados por una secuencia de puntos cercanos, aunque la forma global no sea compacta.
- **No controla la varianza interna:** puede producir clústeres con alta variabilidad o formas no convexas.

Modelos basados en Árboles

(a) ¿Cómo maneja un **árbol de decisión** como **CART** las características **continuas**? Describa el proceso que sigue el algoritmo para encontrar el punto de corte óptimo para una variable numérica.

Respuesta:

Un árbol de decisión del tipo **CART** (*Classification and Regression Trees*) maneja las características **continuas** mediante **cortes univariados** de la forma $x_j \leq t$, donde x_j es una característica numérica y t es un umbral o punto de corte. Cada división separa el espacio en dos regiones: los puntos que cumplen la condición y los que no.

Proceso para determinar el punto de corte óptimo

Para cada característica continua, CART sigue un procedimiento sistemático:

1. **Ordenar los datos según la característica** Para una variable continua x_j , el algoritmo ordena los valores:

$$x_{(1)} \leq x_{(2)} \leq \dots \leq x_{(n)}.$$

2. **Generar candidatos a puntos de corte** Los posibles umbrales t se eligen típicamente como los puntos medios entre pares de valores consecutivos distintos:

$$t_k = \frac{x_{(k)} + x_{(k+1)}}{2}.$$

Esto evita cortes redundantes y reduce la complejidad sin perder generalidad.

3. **Evaluar cada corte candidato** Para cada posible t_k , se divide el conjunto en:

$$R_{\text{izq}}(t_k) = \{\mathbf{x} : x_j \leq t_k\}, \quad R_{\text{der}}(t_k) = \{\mathbf{x} : x_j > t_k\}.$$

Luego se calcula el **criterio de impureza** resultante de esta división:

- Para clasificación: impureza Gini o entropía.
- Para regresión: error cuadrático medio (MSE).

La impureza total de la división se computa como un promedio ponderado:

$$\text{Impureza}(t_k) = \frac{|R_{\text{izq}}(t_k)|}{n} I(R_{\text{izq}}(t_k)) + \frac{|R_{\text{der}}(t_k)|}{n} I(R_{\text{der}}(t_k)).$$

4. **Seleccionar el punto de corte óptimo** El algoritmo elige el valor t_k que **minimiza** la impureza total:

$$t^* = \arg \min_{t_k} \text{Impureza}(t_k),$$

o equivalentemente, el que produce la **mayor ganancia de información**.

(b) La **Ganancia de Información** para un atributo A al dividir un conjunto S se define como: $\text{Gain}(S, A) = I(S) - \sum_{v \in \text{values}(A)} \frac{|S_v|}{|S|} I(S_v)$, donde $I(\cdot)$ es una medida de impureza como la Entropía o Gini. Usando esta fórmula, explique por qué un atributo como **ID de transacción** (con un valor único para cada muestra). ¿Es este modelo útil?

Respuesta:

Supongamos que A es un identificador único de cada muestra (ID de transacción). Entonces:

- Cada valor v aparece exactamente una vez: $|S_v| = 1$.
- Cada subconjunto S_v contiene solo un ejemplo, que por definición tiene **impureza** $I(S_v) = 0$, ya que todos los ejemplos en S_v pertenecen a la misma clase (solo hay uno).

Entonces la ganancia de información se calcula como:

$$\text{Gain}(S, A) = I(S) - \sum_v \frac{1}{|S|} \cdot 0 = I(S) - 0 = I(S),$$

lo que es el **máximo posible** de ganancia de información. Es decir, este atributo sería elegido inmediatamente para dividir el nodo, porque aparentemente reduce toda la impureza.

Por qué no es útil

Aunque la ganancia de información es máxima, dividir por un ID de transacción **no generaliza**:

- Cada nodo hoja contendrá exactamente una muestra; el árbol se ajusta perfectamente a los datos de entrenamiento (**sobreajuste extremo**).
- No captura patrones ni relaciones entre características y variable objetivo.
- Para nuevos datos con IDs distintos, el árbol no puede hacer predicciones útiles.
- En estos casos, además de establecer un **mínimo de muestras por hoja**, otros criterios para regularizar y evitar sobreajuste incluyen:
 - **Profundidad máxima del árbol** (**max_depth**): limita el número de niveles de decisión.
 - **Mínimo de muestras para dividir un nodo** (**min_samples_split**): requiere que un nodo tenga cierta cantidad de ejemplos antes de intentar una división.
 - **Mínimo de impureza disminuida** (**min_impurity_decrease**): solo realiza la división si reduce la impureza en un umbral mínimo.
 - **Máximo número de hojas** (**max_leaf_nodes**): limita la cantidad total de nodos hoja en el árbol.
 - **Poda post-entrenamiento** (*pruning*): eliminar ramas que no aportan mejora significativa en validación para mejorar generalización.

(c) Un **árbol de decisión** particiona el espacio de características en regiones **hiper-rectangulares**. ¿Cómo afecta esta propiedad a su capacidad para modelar relaciones lineales entre las características y la variable objetivo? Por ejemplo, si la verdadera frontera de decisión es una **diagonal** (ej. $x_1 + x_2 > c$), ¿cómo la aproximaría un árbol de decisión? ¿Qué implicación tiene esto en la complejidad del árbol?

Respuesta:

Particionamiento hiper-rectangular

Un árbol de decisión particiona el espacio de características mediante cortes **axis-aligned**, es decir, umbrales de la forma $x_j \leq t$, para cada característica x_j . Cada división crea regiones **hiper-rectangulares** (rectángulos en 2D, cuboides en dimensiones superiores) que definen los nodos hoja.

Aproximación de fronteras diagonales

Supongamos que la frontera de decisión verdadera es diagonal:

$$x_1 + x_2 > c.$$

Un árbol de decisión no puede crear cortes diagonales directamente, por lo que debe aproximar la diagonal mediante una **serie de cortes horizontales y verticales**, generando una forma escalonada ("staircase approximation"):

- Cada corte en x_1 o x_2 define un borde recto paralelo a un eje.
- A medida que se agregan más cortes, la aproximación se vuelve más cercana a la diagonal, pero nunca exactamente igual.
- La frontera resultante en el árbol tiene forma de escalera, lo que introduce errores locales.

Implicaciones para la complejidad del árbol

Para representar correctamente una frontera diagonal:

- Se requieren múltiples cortes, lo que aumenta la **profundidad del árbol**.
- La cantidad de nodos crece, aumentando la **complejidad computacional** y la memoria necesaria.
- Mayor complejidad aumenta el riesgo de **sobreajuste**, especialmente si se añaden nodos para capturar pequeñas variaciones de los datos.

Conclusión

Un árbol de decisión puede aproximar cualquier frontera lineal, pero cuando esta no está alineada con los ejes, la aproximación requiere múltiples cortes hiper-rectangulares. Esto genera un árbol más profundo y complejo, con riesgo de sobreajuste y mayor dificultad para generalizar. En estas situaciones, combinaciones de árboles (como Random Forests o Gradient Boosting) o métodos lineales explícitos pueden modelar la relación más eficientemente.

P2. (20 %) Construcción de Árbol de Decisión

Dado el siguiente conjunto de datos para predecir si una especie de planta florece bajo ciertas condiciones, donde Luz Solar, Humedad y Fertilizante son los atributos, y Planta Florece es la clase objetivo.

Tabla 1: Dataset para predicción de floración de una planta.

Luz Solar	Humedad	Fertilizante	Planta Florece
Directa	Alta	No	Sí
Indirecta	Alta	No	No
Indirecta	Baja	No	No
Directa	Baja	Sí	Sí
Directa	Alta	Sí	Sí
Indirecta	Baja	Sí	No

(a) (1.5 puntos) Calcule la **entropía inicial** del conjunto de datos completo (el nodo raíz) con respecto a la clase **Planta Florece**.

Respuesta:

El conjunto completo tiene 6 ejemplos: 3 con **Sí** y 3 con **No** para *Planta Florece*. La entropía se calcula como:

$$I(S) = - \sum_{c \in \{\text{Sí}, \text{No}\}} p_c \log_2 p_c = - \left(\frac{3}{6} \log_2 \frac{3}{6} + \frac{3}{6} \log_2 \frac{3}{6} \right) = - \left(\frac{1}{2} \log_2 \frac{1}{2} + \frac{1}{2} \log_2 \frac{1}{2} \right) = 1.$$

Por lo tanto, $I(S) = 1$.

(b) (1.5 puntos) Calcule la **ganancia de información (Information Gain)** para cada uno de los tres atributos: **Luz Solar**, **Humedad** y **Fertilizante**.

Respuesta:

1. Luz Solar: Valores: Directa (3 ejemplos: Sí, Sí, Sí), Indirecta (3 ejemplos: No, No, No). - Entropía de cada subconjunto:

$$I(\text{Directa}) = - \left(\frac{3_{SI}}{3} \cdot \log_2 \frac{3_{SI}}{3} + \frac{0_{NO}}{3} \cdot \log_2 \frac{0_{NO}}{3} \right) = 0, \quad I(\text{Indirecta}) = - \left(\frac{0_{SI}}{3} \cdot \log_2 \frac{0_{SI}}{3} + \frac{3_{NO}}{3} \cdot \log_2 \frac{3_{NO}}{3} \right) = 0$$

- Entropía ponderada:

$$I(S|\text{Luz Solar}) = \frac{3}{6} \cdot 0 + \frac{3}{6} \cdot 0 = 0$$

- Ganancia de información:

$$\text{Gain}(S, \text{Luz Solar}) = I(S) - I(S|\text{Luz Solar}) = 1 - 0 = 1$$

2. Humedad: Valores: Alta (3 ejemplos: Sí, No, Sí), Baja (3 ejemplos: No, Sí, No).

- Subconjunto Alta: 2 Sí, 1 No $\Rightarrow I = -\frac{2}{3} \log_2 \frac{2}{3} - \frac{1}{3} \log_2 \frac{1}{3} \approx 0.918$

- Subconjunto Baja: 1 Sí, 2 No $\Rightarrow I \approx 0.918$

- Entropía ponderada:

$$I(S|\text{Humedad}) = \frac{3}{6} \cdot 0.918 + \frac{3}{6} \cdot 0.918 = 0.918$$

- Ganancia:

$$\text{Gain}(S, \text{Humedad}) = 1 - 0.918 \approx 0.082$$

3. Fertilizante: Valores: Sí (3 ejemplos: Sí, Sí, No), No (3 ejemplos: Sí, No, No)

- Subconjunto Sí: 2 Sí, 1 No $\Rightarrow I \approx 0.918$

- Subconjunto No: 1 Sí, 2 No $\Rightarrow I \approx 0.918$

- Entropía ponderada: $I(S|\text{Fertilizante}) = 0.918$ - Ganancia: $\text{Gain}(S, \text{Fertilizante}) = 1 - 0.918 \approx 0.082$

(c) (1.5 puntos) Basado en sus cálculos, ¿qué atributo se seleccionaría como el **nodo raíz** del árbol de decisión según el algoritmo **ID3**? Justifique su respuesta.

Respuesta:

El atributo con mayor ganancia de información se selecciona como **nodo raíz**.

$$\text{Gain}(\text{Luz Solar}) = 1 > \text{Gain}(\text{Humedad}) \approx 0.082, \quad \text{Gain}(\text{Fertilizante}) \approx 0.082$$

$\Rightarrow$ El nodo raíz será Luz Solar, ya que separa perfectamente los ejemplos en clases puras, reduciendo al máximo la impureza del nodo raíz, generando un árbol más compacto. (criterio ID3).

(d) (1.5 puntos) Dibuje el primer nivel del árbol de decisión. Si una de las ramas resultantes conduce a un **nodo puro** (todos los ejemplos pertenecen a la misma clase), indíquelo.

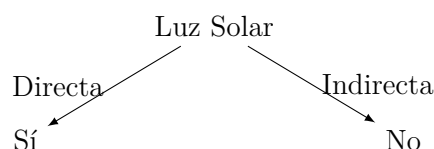
Tabla 2: Tabla 2: Tabla de resultados aproximados.

$$\begin{array}{ll} \log_2(1) = 0 & -\frac{1}{2} \log_2\left(\frac{1}{2}\right) = 0.5 \\ \log_2\left(\frac{1}{2}\right) = -1 & -\frac{2}{3} \log_2\left(\frac{2}{3}\right) \approx 0.390 \\ \log_2\left(\frac{1}{3}\right) \approx -1.585 & -\frac{1}{3} \log_2\left(\frac{1}{3}\right) \approx 0.528 \\ \log_2\left(\frac{2}{3}\right) \approx -0.585 & \end{array}$$

Respuesta:

División por **Luz Solar**:

$$\begin{cases} \text{Directa} & \Rightarrow \text{Sí (nodo puro)} \\ \text{Indirecta} & \Rightarrow \text{No (nodo puro)} \end{cases}$$



P3. (25 %) Kernels y Clustering

Considere el **kernel polinomial** de orden 2, $K : [-5, 5]^2 \rightarrow \mathbb{R}$, definido como $K(x, y) = (x^T y + 1)^2$, donde $[-5, 5]^2$ es el cuadrado en $\mathbb{R}^2$ cuyas coordenadas están entre -5 y 5 .

(a) (1.5 puntos) Pruebe que K es un **kernel de Mercer válido**.

Indicación: le puede ser de utilidad que si se denota

$$X = \begin{pmatrix} x_1 & \dots & x_n \end{pmatrix} \in \mathbb{R}^{2 \times n}, \quad c = (c_1, \dots, c_n)^T \in \mathbb{R}^n,$$

entonces

$$\sum_{i=1}^n \sum_{j=1}^n c_i c_j (x_i^T x_j)^2 = \|Xc\|^2.$$

Respuesta:

Se tiene el kernel polinomial de orden 2:

$$K(x, y) = (x^T y + 1)^2$$

Para probar que es un kernel de Mercer válido, debemos verificar que para cualquier conjunto de puntos $x_1, \dots, x_n \in \mathbb{R}^2$ y cualquier vector $c \in \mathbb{R}^n$ se cumpla:

$$\sum_{i=1}^n \sum_{j=1}^n c_i c_j K(x_i, x_j) \geq 0.$$

Expandimos K :

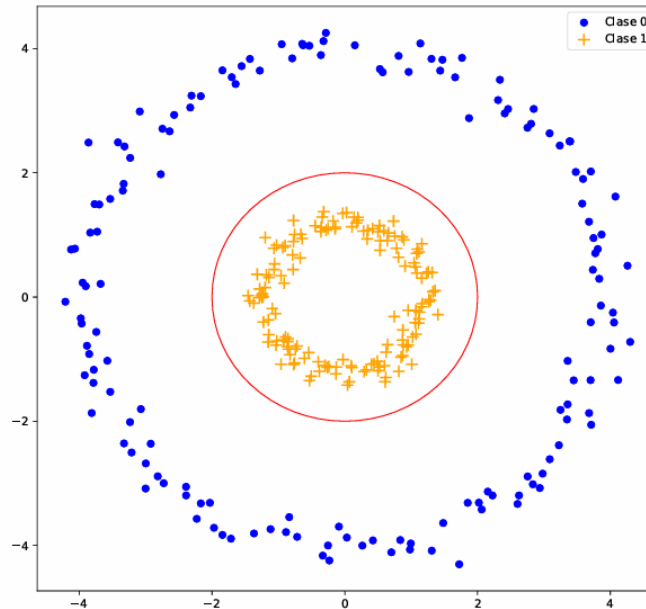
$$K(x_i, x_j) = (x_i^T x_j + 1)^2 = (x_i^T x_j)^2 + 2x_i^T x_j + 1.$$

Usando la indicación:

$$\sum_{i,j} c_i c_j (x_i^T x_j)^2 = \|Xc\|^2 \geq 0,$$

y de manera similar, $\sum_{i,j} c_i c_j x_i^T x_j = \|X^T c\|^2 \geq 0$ y $\sum_{i,j} c_i c_j = (\sum_i c_i)^2 \geq 0$. Como suma de cantidades no negativas, K cumple la condición de Mercer. $\Rightarrow K$ es un kernel válido.

Considere de ahora en adelante un conjunto de datos $X = \{(\vec{x}_i, y_i)\}_{i=1}^N$ como el de la siguiente imagen, en donde en forma de círculos se encuentran los puntos de la clase 0, en cruces los de la clase 1 y en línea sólida un círculo centrado en 0 de radio 2.



(b) (1 punto) Entregue una posible solución que entregaría **k-Means a 2 clústers** para agrupar este dataset, argumentando su respuesta. ¿Recupera este algoritmo la estructura inicial? ¿por qué?

Respuesta:

El dataset consiste en **dos círculos concéntricos**, donde la clase 0 es el círculo externo y la clase 1 el interno.

- **k-Means** busca minimizar la distancia euclidiana al centroide de cada clúster.
- En este caso, el algoritmo tenderá a dividir los puntos en dos grupos basados en la posición *horizontal y vertical*, generando probablemente clústers que no correspondan a los círculos concéntricos. Por ejemplo, la mitad izquierda de la visualización será el cluster 0 y la mitad derecha el cluster 1.
- **Conclusión:** k-Means *no recupera la estructura original*, porque asume clústers convexos y separados por hipersferas, mientras que aquí la separación es **anular**.

(c) (1 punto) Considere la función $\phi(\mathbf{x}, \mathbf{y}) = (x^2, y^2, 1)$, ¿por qué es relevante la definición de esta función en el contexto de un dataset como este?

Respuesta:

La función de transformación ϕ mapea los puntos 2D a un espacio 3D:

$$(x, y) \mapsto (x^2, y^2, 1)$$

- Transforma los círculos concéntricos en grupos aproximadamente **linealmente separables** en 3D.
- En este espacio, los puntos de clase 0 y 1 pueden separarse mediante un plano, lo que permite a k-Means encontrar clústers correctamente.

- Esto es un ejemplo de la **técnica de kernel trick** para resolver problemas no lineales.

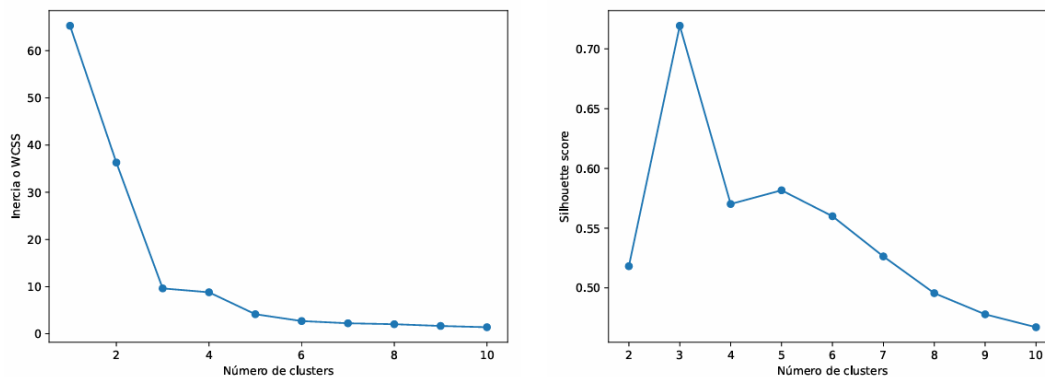
(d) (1.5 puntos) Se plantea hacer **k-Means con 2 clústers** en el nuevo dataset dado por transformar el original por ϕ , digamos $Z = \{\phi(x_i^1, x_i^2)\}_{i=1}^N$, ¿qué ventajas puede tener esto en el nuevo dataset con respecto al original? Esboce la respuesta del algoritmo en un espacio tridimensional.

Respuesta:

Ventajas:

- Los clústers se vuelven **convexos y esféricos** en el nuevo espacio, cumpliendo los supuestos de k-Means.
- Se mejora la separación entre las clases; el algoritmo puede asignar correctamente los puntos de los círculos interno y externo.
- El algoritmo en 3D aproximará un plano que divide los dos conjuntos: $z_3 = 1$ o alguna combinación lineal de z_1, z_2 creando dos clústers claramente separados.

(e) (1 punto) Para decidir una mejor cantidad de clústers, se realizan análisis de **Inercia (método del codo)** y **coeficiente de Silueta**, cuyos gráficos quedan a continuación. Según esto, ¿cuántos clústers debería elegir? ¿es problemático esto con la estructura original del dataset? De ser problemático, ¿a qué falencia de los métodos vistos en el curso se puede atribuir?



Respuesta:

De los datos proporcionados:

- Método del codo (Inercia/WCSS): gran caída hasta $K = 2$, luego disminución más lenta.
- Coeficiente de Silueta: máximo en $K = 3$ (Silhouette ≈ 0.71).

Interpretación:

- Según Inercia, se sugiere $K = 2$, que coincide con el número de círculos originales.
- Según Silueta, el valor máximo ocurre en $K = 3$, lo que podría indicar **subestructuras locales** o la limitación de k-Means de suponer clústers convexos.

Problema: La discrepancia surge porque k-Means y métricas como Silueta asumen clústers convexos y separados por distancias euclidianas, mientras que el dataset original tiene clústers **concéntricos y no convexos**. Por lo tanto, algunos métodos pueden sugerir más clústers de los necesarios debido a esta **falencia de los algoritmos basados en centroides** para datos anulares o con forma arbitraria.